E-commerce Store: Complete Project Plan

FalconXoft Weekend 1 Assignment | Project 2 of 3 | Suggested stack: Django + Next.js + PostgreSQL (same as the Blog project)

This document explains everything you must plan, build, test and submit for the E-commerce project, based on the assignment PDF. It is written in simple English so you can also use it to prepare for the Monday interview.

1. What the PDF asks for (E-commerce project)

Minimum functional scope: product catalog, cart, stock control, mock-payment checkout, orders and order status.

Key rules from the PDF:

- Build a working MVP with one complete end-to-end user journey (browse, add to cart, checkout, order).
- Payment is **mock only**. Do not connect a real payment gateway. Be honest in the README that payment is simulated.
- Never say something is done if it is not done. Write what is missing and why.
- Every decision must be defensible in the interview. The reviewer can ask for a walkthrough, debugging or a small code change.
- AI use is not encouraged. If you use it, disclose the tool, the areas and how you verified the result.
- No passwords, keys or secrets in the repo or Drive folder.

Scoring (per project): Problem understanding 20%, Architecture and design 25%, Implementation 25%, Testing 20%, Explanation and ownership 10%.

Why this project is different from the Blog: the hard part here is not showing pages, it is **keeping data correct**. Stock must never go below zero, two buyers must not buy the last item at the same time, and a repeated payment click must not create two orders. The reviewer will test these points.

2. Requirements and user flow

2.1 Business goal

A simple online shop where customers browse products, add them to a cart, pay (mock) and track their orders, and the shop admin manages products, stock and order status.

2.2 Users and roles

Role	Who they are	What they can do
Visitor (not logged in)	Anyone	Browse and search products, view product details, register, log in
Customer	Registered user (default role)	Everything a visitor can do, plus use a cart, checkout, see own orders, cancel own order (if still allowed)
Admin	Shop manager	Manage products, categories and stock, see all orders, change order status

Assumption: the cart is only for logged-in customers (simple and safer). A guest cart is listed as a future improvement.

2.3 Scope

In scope (must build):

1. Register, login, logout (JWT)
2. Product catalog with categories, images (URL field), price, stock
3. Search and filter products (by name, category, price range) with pagination
4. Cart: add item, change quantity, remove item, view total
5. Stock control: stock check at add-to-cart and again at checkout; stock reduces after a successful order
6. Mock-payment checkout: a fake payment step that can succeed or fail
7. Orders: create order, view order history, order details
8. Order status: pending, paid, shipped, delivered, cancelled (admin updates status)
9. Admin panel: product CRUD, stock update, order list and status update

Out of scope (say this honestly in README):

- Real payment gateway (Stripe, JazzCash, etc.)
- Email or SMS notifications (proposed, needs a service)
- Shipping cost calculation, tax rules, coupons and discounts
- Product variants (size, color), product reviews
- Real image upload to cloud storage (use image URL)

2.4 Assumptions

- One currency (for example PKR or USD, choose one and write it in the README). Prices are stored as whole minor units or DECIMAL, never as float.
- One warehouse, one shipping address per order, no shipping fee.
- Stock is reduced when payment succeeds. (Alternative: reserve stock when the cart is checked out. You must explain your choice.)
- Prices in the order are copied at purchase time, so later price changes do not change old orders.
- Mock payment: a fake "card token" where a special value (for example `fail`) forces a failure, so you can test the failure path.

2.5 Main user journey (end to end)

1. Customer registers and logs in.
2. Customer browses the catalog, searches for a product and opens its detail page.
3. Customer adds two items to the cart and changes one quantity.
4. Customer opens the cart and sees the correct total.
5. Customer clicks Checkout, enters a shipping address and pays with mock payment.
6. Payment succeeds, order is created with status Paid, stock is reduced, cart is emptied.
7. Customer opens My Orders and sees the new order.
8. Admin logs in, sees the order, and changes the status to Shipped, then Delivered.
9. Failure path: customer tries to buy more than available stock, or the mock payment fails. The user sees a clear error and no stock is lost.

2.6 Acceptance criteria

ID	Criteria
AC1	A customer can register, log in and see only their own cart and orders
AC2	Products can be listed, searched and filtered; out-of-stock products are clearly shown
AC3	Cart quantity cannot be zero, negative or more than available stock
AC4	Cart total is always correct (sum of price x quantity)
AC5	Checkout creates exactly one order, even if the request is repeated
AC6	Stock goes down after a successful order and never goes below zero
AC7	A failed payment creates no paid order and does not reduce stock
AC8	Only admin can create or edit products and change order status
AC9	Order status can only move in allowed steps (for example, Delivered cannot go back to Pending)
AC10	The project runs from the README steps on a clean machine

3. Technology stack

Layer	Choice	Why (simple reason)
Backend	Django + Django REST Framework	Strong ORM, transactions, built-in auth and admin
Frontend	Next.js (React)	Good for catalog pages and SEO, easy routing

Layer	Choice	Why (simple reason)
Database	PostgreSQL	Needs transactions and row locking to protect stock and orders
Auth	JWT with djangorestframework-simplejwt	Works with a separate frontend
Payments	Mock payment service (own code)	Required by the PDF; no real money
Containers	Docker + docker-compose	Same setup on every machine
Tests	pytest + pytest-django	Easy to write permission, stock and duplicate tests
CI/CD	GitHub Actions	Run tests on every push
API docs	drf-spectacular (openapi.yaml)	Satisfies the api-spec requirement

Alternatives you should be able to explain:

- *MongoDB:* orders, stock and payments need strong consistency and transactions. SQL is a better fit.
- *Store cart in browser only (localStorage):* simple, but stock checks and total price must be trusted from the server. A server-side cart is safer.
- *Store cart in Redis:* fast, but extra service. A cart table in PostgreSQL is enough for an MVP.
- *Real gateway (Stripe):* not required, adds keys and risk. Mock payment keeps the MVP simple but must have a clear interface so a real gateway can replace it.

4. Architecture

4.1 Components

1. **Browser:** the customer or admin.
2. **Next.js frontend:** catalog, cart, checkout and order pages.
3. **Django REST API:** business rules (stock, totals, order status, permissions).
4. **PostgreSQL:** users, products, carts, orders, payments.
5. **Mock payment service:** a separate module inside Django (clean interface `charge(order, token)`) that returns success or failure.

6. **Redis (proposed):** cache for product lists and rate limiting.
7. **Background worker with Celery (proposed):** order confirmation emails, cancelling unpaid orders after a timeout, low-stock alerts.
8. **Nginx reverse proxy (proposed):** HTTPS and routing.

4.2 Data movement

- Browser calls Nginx. Nginx sends pages to Next.js and `/api/` to Django.
- Django checks the JWT and role, then reads or writes PostgreSQL.
- At checkout, Django opens a database transaction: lock stock rows, create the order, call the mock payment, reduce stock, save the payment, empty the cart, commit. If anything fails, it rolls back.

4.3 Deployment boundaries

Public zone: Nginx only. Private zone: Next.js, Django, PostgreSQL, Redis. Only ports 80 and 443 are public. The mock payment lives inside the private zone.

4.4 Implemented vs proposed

Item	Status
Django API, PostgreSQL, Next.js, JWT auth	Implemented
Cart, stock control, mock payment, orders	Implemented
Docker and docker-compose	Implemented
GitHub Actions tests	Implemented (if finished)
Nginx and HTTPS	Proposed
Redis cache	Proposed
Celery workers (emails, auto-cancel)	Proposed
Real payment gateway	Proposed (not built)
Monitoring (Sentry, uptime)	Proposed

Draw this in Lucidchart for architecture.pdf. Label every arrow (for example "HTTPS JSON", "SQL over private network") and draw boxes for the public and private zones. Include the mock payment as its own box so the reviewer sees where a real gateway

would connect.

5. Folder structure

5.1 Code repository

```
ecommerce-store/
├── README.md
├── docker-compose.yml
├── .env.example              (variable names only)
├── .gitignore
├── .github/workflows/ci.yml
├── backend/
│   ├── Dockerfile
│   ├── manage.py
│   ├── requirements.txt
│   ├── config/               (settings, urls, wsgi)
│   ├── apps/
│   │   ├── users/            (auth, roles)
│   │   ├── catalog/          (Category, Product, stock)
│   │   ├── cart/             (Cart, CartItem)
│   │   ├── orders/           (Order, OrderItem, status rules, checkout)
│   │   └── payments/         (mock payment service, Payment model)
│   └── tests/
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   ├── app/                  (home, products, product/[id], cart, checkout,
orders, admin)
│   ├── components/
│   └── lib/api.js
└── docs/
    ├── api-spec/openapi.yaml
    └── schema.sql
```

5.2 Drive submission folder (this project)

```
Weekend1_YourName/
├── 01_Blog/
├── 02_Ecommerce/
|   ├── 01-README.md
|   ├── 02-architecture.pdf
|   ├── 03-database-design.pdf  (+ schema.sql)
|   ├── 04-api-workflows.pdf    (+ api-spec.md or openapi.yaml)
|   ├── 05-test-cases.xlsx
|   ├── 06-engineering-decisions.md
|   ├── 07-source-and-demo-links.md
|   ├── 08-cost-and-scaling.xlsx
|   └── 09-sources-and-disclosure.md
└── 03_Marketplace/
```

(The `submission-index.md` stays in the parent folder and lists all three projects.)

6. Database design

6.1 Tables

users

Field	Type	Notes
id	BIGSERIAL	Primary key
email	VARCHAR(254)	Unique, not null
password_hash	VARCHAR(255)	Hashed
full_name	VARCHAR(120)	Not null
role	VARCHAR(10)	Check: customer, admin. Default customer
is_active	BOOLEAN	Default true
created_at	TIMESTAMPTZ	Default now

categories

Field	Type	Notes
id	BIGSERIAL	Primary key
name	VARCHAR(80)	Unique, not null
slug	VARCHAR(100)	Unique, not null

products

Field	Type	Notes
id	BIGSERIAL	Primary key
category_id	BIGINT	FK to categories.id, ON DELETE PROTECT
name	VARCHAR(200)	Not null
slug	VARCHAR(220)	Unique
description	TEXT	
price	NUMERIC(10,2)	Check: price >= 0
stock_quantity	INTEGER	Check: stock_quantity >= 0
image_url	VARCHAR(500)	
is_active	BOOLEAN	Default true (hide instead of delete)
created_at, updated_at	TIMESTAMPTZ	Auto

carts

Field	Type	Notes
id	BIGSERIAL	Primary key
user_id	BIGINT	FK to users.id, unique (one active cart per user)
updated_at	TIMESTAMPTZ	Auto

cart_items

Field	Type	Notes
id	BIGSERIAL	Primary key
cart_id	BIGINT	FK to carts.id, ON DELETE CASCADE
product_id	BIGINT	FK to products.id
quantity	INTEGER	Check: quantity > 0
		Unique (cart_id, product_id) so the same product is not added twice as two rows

orders

Field	Type	Notes
id	BIGSERIAL	Primary key
user_id	BIGINT	FK to users.id
status	VARCHAR(12)	Check: pending, paid, shipped, delivered, cancelled
total_amount	NUMERIC(12,2)	Check >= 0
shipping_address	TEXT	Not null
idempotency_key	VARCHAR(64)	Unique per user, used to stop duplicate orders
created_at, updated_at	TIMESTAMPTZ	Auto

order_items

Field	Type	Notes
id	BIGSERIAL	Primary key
order_id	BIGINT	FK to orders.id, ON DELETE CASCADE
product_id	BIGINT	FK to products.id, ON DELETE PROTECT
product_name	VARCHAR(200)	Copy at purchase time

Field	Type	Notes
unit_price	NUMERIC(10,2)	Copy at purchase time
quantity	INTEGER	Check > 0

payments

Field	Type	Notes
id	BIGSERIAL	Primary key
order_id	BIGINT	FK to orders.id
amount	NUMERIC(12,2)	
status	VARCHAR(10)	Check: success, failed
provider_ref	VARCHAR(64)	Fake reference from the mock
created_at	TIMESTAMPTZ	Auto

order_status_history (optional, nice to have)

Field	Type	Notes
id	BIGSERIAL	Primary key
order_id	BIGINT	FK to orders.id
old_status, new_status	VARCHAR(12)	
changed_by	BIGINT	FK to users.id
changed_at	TIMESTAMPTZ	

6.2 Relationships (cardinality)

- One category has many products.
- One user has one active cart; one cart has many cart items; each cart item points to one product.
- One user has many orders; one order has many order items; each order item points to one product.

- One order has one or more payment attempts.

6.3 Indexes

Index	Why
products(slug) unique	Product page lookup
products(category_id, is_active)	Category listing
products(price)	Price filter and sorting
GIN or trigram index on products(name)	Search by name
cart_items(cart_id, product_id) unique	Prevent duplicate rows
orders(user_id, created_at DESC)	My Orders page
orders(status)	Admin filter by status
orders(user_id, idempotency_key) unique	Duplicate checkout protection
order_items(order_id)	Load order details

6.4 Data integrity points (very important here)

- **CHECK constraint stock_quantity >= 0:** the database itself refuses negative stock, even if a code bug exists.
- **Row locking:** at checkout use `SELECT ... FOR UPDATE` (Django: `select_for_update()`) on the product rows inside a transaction, so two buyers cannot take the last item together.
- **Transactions:** order, order items, payment record, stock reduction and cart clearing succeed together or fail together.
- **Price snapshot:** copy product name and price into order_items.
- **Idempotency key:** the same checkout request sent twice returns the same order, not a second one.
- **No hard delete of products** that appear in orders. Use is_active = false.
- **Money type:** always NUMERIC/DECIMAL, never float.

Deliverables: ERD in Lucidchart (cardinality, PK, FK, types, indexes) exported as database-design.pdf, plus `schema.sql` or Django migrations.

7. API design

Base URL: `/api/v1/`. Format: JSON. Auth header: `Authorization: Bearer <access_token>`.

7.1 Routes

Method	Route	Auth	Purpose
POST	/auth/register/	Public	Create account
POST	/auth/login/	Public	Get tokens
POST	/auth/refresh/	Public (refresh token)	New access token
GET	/auth/me/	Logged in	Profile
GET	/categories/	Public	List categories
GET	/products/	Public	List products (?search=, ?category=, ?min_price=, ?max_price=, ?page=)
GET	/products/{id}/	Public	Product detail
POST, PATCH, DELETE	/products/ and /products/{id}/	Admin	Manage products (delete = deactivate)
PATCH	/products/{id}/stock/	Admin	Update stock
GET	/cart/	Customer	View cart with totals
POST	/cart/items/	Customer	Add item {product_id, quantity}
PATCH	/cart/items/{id}/	Customer	Change quantity
DELETE	/cart/items/{id}/	Customer	Remove item
POST	/checkout/	Customer	Create order and pay (needs Idempotency-Key header)
GET	/orders/	Customer	My orders

Method	Route	Auth	Purpose
GET	/orders/{id}/	Owner or Admin	Order detail
POST	/orders/{id}/cancel/	Owner (only if allowed) or Admin	Cancel order and return stock
GET	/admin/orders/	Admin	All orders (?status=)
PATCH	/admin/orders/{id}/status/	Admin	Change status

7.2 Example: checkout

Request:

```
POST /api/v1/checkout/
Authorization: Bearer <token>
Idempotency-Key: 7f3c9a10-aaaa-4c2d-bb11-0d1e2f3a4b5c
{
  "shipping_address": "House 10, Street 5, Lahore",
  "payment": {"method": "mock_card", "token": "tok_success"}
}
```

Success response (201):

```
{
  "order_id": 42,
  "status": "paid",
  "total_amount": "5400.00",
  "items": [
    {"product_id": 3, "name": "USB Cable", "unit_price": "400.00",
"quantity": 2},
    {"product_id": 9, "name": "Headphones", "unit_price": "4600.00",
"quantity": 1}
  ],
  "payment": {"status": "success", "provider_ref": "mock_8841"}
}
```

Failure response (402) when mock payment fails (token `tok_fail`):

```
{"detail": "Payment failed. No money was taken and your cart is unchanged."}
```

Failure response (409) when stock is not enough:

```
{"detail": "Not enough stock for: Headphones (available: 0)."}
```

7.3 Error cases to design and test

Status	When
400	Invalid input: quantity 0 or negative, empty address, empty cart at checkout
401	Missing or expired token
402	Mock payment failed
403	Customer tries admin route; customer opens another user's order
404	Product or order not found (also used for other users' orders to hide existence)
409	Not enough stock; invalid order status change; duplicate conflict
429	Too many requests (proposed throttling)

7.4 Order status rules

From	Allowed next status
pending	paid, cancelled
paid	shipped, cancelled
shipped	delivered
delivered	none (final)
cancelled	none (final)

When an order is cancelled from paid, stock is returned inside a transaction. Any other jump returns 409.

7.5 Sequence diagrams to draw (04-api-workflows.pdf)

1. **Success path:** Customer logs in, adds to cart, checkout with Idempotency-Key: Django starts a transaction, locks products, creates the order, calls mock payment (success), reduces stock, clears cart, commits, returns 201.
2. **Failure path:** Checkout where mock payment fails or stock is not enough: Django rolls back, returns 402 or 409, stock and cart unchanged. Also show the duplicate request returning the same order.

Also provide `openapi.yaml` (generated by drf-spectacular and checked by you) or `api-spec.md`.

8. Frontend pages (Next.js)

Page	Route	Who sees it
Home and product list with search, filters, pagination	/ and /products	Everyone
Product detail	/products/[slug]	Everyone
Login and Register	/login, /register	Visitors
Cart	/cart	Customer
Checkout (address and mock payment)	/checkout	Customer
Order success and My Orders	/orders, /orders/[id]	Customer
Admin: products and stock	/admin/products	Admin
Admin: orders and status	/admin/orders	Admin

Frontend rules: protect cart, checkout and admin routes; show loading, empty-cart and error states; disable the Pay button after the first click and send the same Idempotency-Key on retry; never calculate the final price on the frontend only (the backend always recalculates).

9. Implementation plan (build order)

1. Setup: Docker, PostgreSQL, Django, Next.js, .env.example. Reuse your own auth and Docker setup from the Blog project (mention it in the disclosure file).

2. Catalog: Category and Product models, admin CRUD, list, search, filters.
3. Cart: add, update, remove, totals, stock checks.
4. Orders and checkout: transaction, row locking, idempotency key, price snapshot.
5. Mock payment service with success and fail tokens.
6. Order status rules and admin order management; cancel with stock return.
7. Frontend pages in this order: login, catalog, product, cart, checkout, orders, admin.
8. Tests along the way, especially checkout, stock and permissions.
9. README, diagrams, cost sheet, decisions, disclosure, demo video.

Environment variable names (no values): DJANGO_SECRET_KEY, DJANGO_DEBUG, DJANGO_ALLOWED_HOSTS, DATABASE_URL (or POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_HOST, POSTGRES_PORT), JWT_ACCESS_LIFETIME_MINUTES, JWT_REFRESH_LIFETIME_DAYS, CORS_ALLOWED_ORIGINS, NEXT_PUBLIC_API_URL, MOCK_PAYMENT_ENABLED, CURRENCY_CODE.

10. Testing plan (05-test-cases.xlsx)

Required columns: Test ID, Feature, Preconditions, Steps, Input, Expected Result, Actual Result, Status, Defect ID, Evidence Link.

Run every test and write the real Actual Result. Attach pytest output or screenshots as evidence. Do not mark a test as Pass without running it.

Type	Example tests
Positive	Register, login, add to cart, checkout success, order appears in history, admin ships order
Negative	Wrong password, add a missing product, checkout with empty cart, payment token `tok_fail`
Boundary	Quantity 0, 1, exact available stock, stock plus 1, very large quantity, price 0, 2 decimal places, page size limits
Permissions	Customer calls admin routes (403), customer opens another user's order (404), visitor opens cart (401), customer changes order status (403)

Type	Example tests
Duplicate and retry	Same Idempotency-Key sent twice gives one order; double-click on Pay; add same product twice merges quantity; cancel an already cancelled order; retry after payment failure works
Stock and concurrency	Two checkouts for the last item at the same time: only one succeeds, stock never below zero
Status rules	Delivered to Pending is rejected (409); cancel from paid returns stock
Regression	After each bug fix, re-run the full suite and add a test for that bug
Data integrity	Price change after purchase does not change the old order; negative stock rejected by the database

Aim for 35 to 45 well-written tests. The concurrency and idempotency tests are the ones that show real understanding, so do not skip them.

11. Deployment and cost analysis (08-cost-and-scaling.xlsx)

11.1 Two hosting options to compare

Option A: VPS (for example Hostinger or similar)

- One VPS running Docker: Nginx, Next.js, Django (gunicorn), PostgreSQL, optional Redis.
- Cheapest and simple. You manage backups, security updates and monitoring.

Option B: AWS

- Compute with EC2 or ECS, database with RDS PostgreSQL, S3 for product images and backups, CloudFront CDN, Route 53 DNS, CloudWatch monitoring, ElastiCache for Redis, SQS or Celery workers for background jobs.
- Costs more but is managed and easier to scale.

11.2 Cost lines to include for each option

Compute, database, storage (product images), backups, bandwidth and egress, monitoring, domain name, third-party services (email, error tracking, real payment fees as a note), and worker costs.

11.3 How to fill the spreadsheet

One table per option with columns: Service, Specification, Region, Usage assumption, Unit price, Monthly estimate, Source URL, Access date. Add totals and scenario totals for 100, 1,000, 10,000 and 1,000,000 users.

Very important rules:

- Open the official pricing pages yourself, copy the price, the URL and the date you checked.
- Mark each number as **Verified** (from a pricing page) or **Assumption** (your estimate).
- State the region and the currency (and the exchange rate if you convert).
- Do not invent prices. If you cannot verify, label it an assumption.
- E-commerce note: a real payment gateway charges a fee per transaction. You use a mock payment, so write the gateway fee as a proposed future cost, not as part of the MVP.

11.4 Docker, CI/CD, queues, caches, security, observability

Topic	What to explain	Status
Docker	Dockerfiles for backend and frontend, docker-compose with PostgreSQL	Implemented
CI/CD	GitHub Actions runs tests and lint; auto-deploy on merge	Tests implemented, deploy proposed
Queues	Celery and Redis for emails, abandoned-order cleanup, low-stock alerts	Proposed
Caches	Redis cache for product list and categories	Proposed

Topic	What to explain	Status
Security	Password hashing, JWT expiry, role checks on every route, input validation, ORM against SQL injection, CORS limits, HTTPS, rate limiting on login and checkout, server-side price calculation, secrets in env variables, never store real card data	Mostly implemented, HTTPS and rate limiting proposed
Observability	Application logs, order and payment logs, error tracking with Sentry, uptime checks, DB backup alerts	Logs implemented, rest proposed

12. Scaling plan

State your traffic assumptions. Example: about 10 percent of users are active per day, each makes around 30 requests per day, 85 percent of traffic is browsing, 2 percent of visits end in an order.

Users	What changes	Main bottleneck	Rough approach
100	Nothing special	None	One small VPS, daily database backup
1,000	Product list caching, tune indexes, move images to object storage	Slow product queries	Stronger VPS, optional Redis
10,000	Separate database from app, several app containers behind a load balancer, CDN for images and static files, Celery workers for emails and cleanup, connection pooling	Database CPU and checkout row locks on popular products	Managed DB, Redis, read replica for browsing

Users	What changes	Main bottleneck	Rough approach
1,000,000	Autoscaling containers, read replicas, move search to a search engine, queue-based order processing for flash sales, stock reservation system, partition or archive old orders, multi-zone setup, CDN for most pages, full monitoring and alerts	Hot products (many buyers on one stock row), database size, payment spikes	Managed cloud; caching and CDN control cost

Explain for each level: database changes, storage changes, queue changes and the monthly cost from your spreadsheet. Be ready to explain the **flash sale problem**: many people buying the same product at once makes row locks a bottleneck.

13. Engineering decisions (06-engineering-decisions.md)

For each major choice write: Options considered, Final choice, Why, Trade-offs.

Suggested topics: reduce stock at payment success vs reserve at checkout start, row locking vs optimistic locking, server cart vs browser cart, idempotency key design, DECIMAL for money, soft delete for products, JWT storage, mock payment interface design, SQL vs NoSQL, one repo vs two, hosting choice.

Also include:

- **Security:** what you protected and how.
- **Technical risks:** overselling stock, double charging, price tampering from the frontend, abandoned pending orders, hot products at scale.
- **Implemented vs planned services:** the table from section 4.4.
- **Known gaps and remediation:** every missing feature, why, and how you would fix it.

14. Open-source and AI disclosure (09-sources-and-disclosure.md)

- **Open source:** list every library and any template or repo you copied from, with URL, license, what you reused, what you changed and what you wrote yourself. List normal packages (Django, DRF, simplejwt, Next.js) with their licenses.

- **Reuse of your own Blog code:** write that auth, roles and Docker setup came from your own Blog project.
- **AI use:** the PDF says AI is not encouraged. If you used any AI tool, write the tool, the assisted areas and how you verified the result. If none, write "None".
- **Pricing citations:** every pricing page URL with access date.
- Only submit code you fully understand. The reviewer may ask you to change checkout or stock logic live.

15. README outline (01-README.md)

1. Project name and purpose
2. Tech stack
3. Features done and not done (state clearly that payment is mock)
4. Assumptions (currency, stock rule, one warehouse)
5. Architecture summary with a small diagram
6. Setup: clone, copy .env.example to .env, `docker compose up`, run migrations, create admin, load sample products
7. How to run tests
8. Mock payment tokens (`tok_success` , `tok_fail`) and how to create demo accounts (no real credentials)
9. Known gaps
10. Links: Lucidchart, demo video, Drive folder

16. Demo video script (3 to 5 minutes)

1. 20 seconds: project intro and stack.
2. 40 seconds: architecture and ERD.
3. 2 minutes: live journey (register, search, add to cart, checkout, order appears, admin ships it).
4. 40 seconds: failure demo (payment fails or not enough stock, stock stays unchanged).
5. 20 seconds: show duplicate checkout returning the same order.
6. 30 seconds: run tests and show results; mention known gaps.

17. Final checklist for the E-commerce project

#	Deliverable	Done?
1	README.md	
2	architecture.pdf and Lucidchart editable link	
3	database-design.pdf, ERD link, schema.sql	
4	api-workflows.pdf, openapi.yaml or api-spec.md	
5	test-cases.xlsx with actual results and evidence	
6	engineering-decisions.md	
7	GitHub repo and 3 to 5 minute demo link	
8	cost-and-scaling.xlsx with sources and dates	
9	sources-and-disclosure.md	
10	E-commerce entry in submission-index.md; reviewer access given	

18. Suggested time plan

Block	Work
1	Requirements, ERD, architecture, API plan (Lucidchart)
2	Backend: catalog, cart, checkout transaction, mock payment, order status
3	Frontend pages and API connection
4	Tests (especially stock, duplicate and permission tests) and bug fixing
5	Cost sheet, scaling, decisions, disclosure, README, demo video

Tip: the checkout logic is the heart of this project. Finish and test it first, before polishing the frontend.

19. Interview preparation

Be ready to answer these without notes:

- What happens if two customers buy the last item at the same time? Show me the code.
- What is an idempotency key and what happens if the user clicks Pay twice?
- Why is price stored in order_items and not read from products?
- When exactly does stock go down and why did you choose that moment?
- Why NUMERIC and not float for money?
- Why do you recalculate the total on the server?
- What happens if payment succeeds but the database fails after it? (Be honest: with a mock payment this is simulated; with a real gateway you would need payment confirmation webhooks and reconciliation.)
- What would break first at 10,000 users, and at a flash sale?
- Make a small change live: for example, add a maximum quantity per order or a new status rule.

20. Next step

Tell me what you want next: the same kind of document for the third project (Two-sided Marketplace), or detailed help on one part of this project such as the ERD and schema.sql, the checkout transaction logic, the test case sheet or the cost sheet structure.